

1. Executive Summary

Core Topic: A comprehensive exploration of Spring XML-Based Configuration, a legacy yet foundational approach to defining, managing, and wiring Beans within the Spring Framework.

Main Takeaway: While modern Spring Boot relies heavily on Annotation-based configuration (e.g., `@Component`, `@Autowired`), mastering XML configuration reveals the raw mechanics of the Inversion of Control (IoC) container. It strips away the "magic" of annotations, proving that the IoC container strictly operates on Configuration Metadata—regardless of how it is provided. Understanding this empowers you to confidently maintain legacy enterprise systems, manage hybrid codebases, and deeply appreciate the automated heavy lifting done by modern Spring Boot.

2. Chronological Deep-Dive

The Shift from Annotations to XML Configuration

In previous stages of learning the Spring Framework, the standard approach to managing objects (Beans) was through **Annotation-Based Configuration**. This involved sprinkling annotations like `@Component`, `@Bean`, and `@ComponentScan` directly across Java classes.

However, there is an older, legacy approach: **XML-Based Configuration**.

Why learn this if it's legacy?

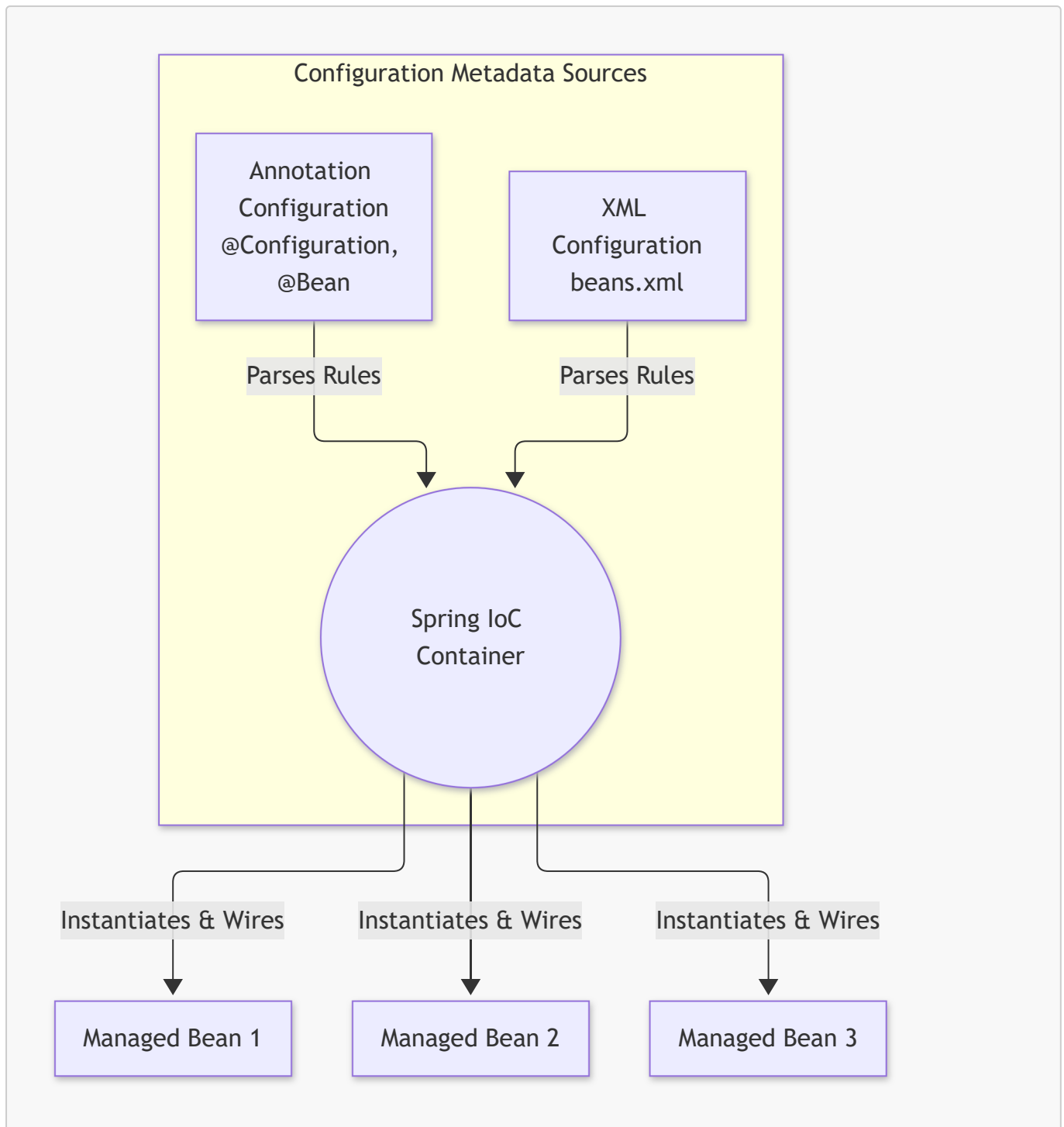
1. **Legacy Systems:** Many older enterprise applications still run heavily on XML configurations.
2. **Hybrid Systems:** Large-scale migrations often result in codebases that mix both annotations and XML.
3. **Deep Structural Understanding:** It forces you to look at how Spring fundamentally operates under the hood. When you understand how tedious it was to write manual XML tags to wire dependencies, you develop a profound respect for Spring Boot's modern auto-configuration capabilities.
4. **Code Purity:** XML configuration removes Spring-specific annotations from your actual Java classes, leaving you with Pure Old Java Objects (POJOs) completely unaware of the Spring framework.

The IoC Container and Configuration Metadata

At the heart of the Spring Framework beats the **IoC (Inversion of Control) Container**. Its primary job is to create, wire, manage, and destroy objects (Beans).

But the IoC container cannot do this blindly. It requires instructions. These instructions are known as **Configuration Metadata**.

The Analogy: Imagine the IoC container is a highly skilled factory manager. The manager is perfectly capable of building and assembling complex products (Beans), but they need a blueprint (Metadata) to know *what* to build and *how* things connect. The manager does not care if the blueprint is drawn digitally on a tablet (Annotations) or drafted manually on physical paper (XML). As long as the instructions are clear, the factory will produce the exact same result.



Project Setup and Maven Dependencies

To experiment with this, we first establish a clean slate. We create a basic Java Maven project (e.g., `xml-based-config-demo`).

Because we want to use the core fundamentals of the IoC container, we must add the `spring-context` dependency. This is fetched from the Maven Central Repository and added directly to the `pom.xml` file inside the `<dependencies>` tag. Once Maven reloads, the Spring core libraries are pulled into our project.

Purging Annotations: Preparing for XML

To truly appreciate the XML approach, we must first strip away everything related to the annotation approach.

Imagine we have a simple business class: `OrderService` with a basic method `placeOrder()` that prints "Order Placed" to the console.

The Teardown Steps:

1. **Remove `@Component`:** We delete the `@Component` annotation from the top of the `OrderService` class. It is now a completely plain Java class.
2. **Delete the Configuration Class:** We entirely delete the `AppConfig` Java class. This was the class annotated with `@Configuration` and `@ComponentScan` that used to tell Spring where to look for beans.
3. **Update the Main Application Runner:** In our `Main` class, we previously booted up the IoC container using `AnnotationConfigApplicationContext`. Because we deleted the `AppConfig` class, this will no longer work.

Bootstrapping the XML Application Context

To tell the IoC container that we are now providing our blueprints via XML, we must change the class used to instantiate the context.

We switch from `AnnotationConfigApplicationContext` to `ClassPathXmlApplicationContext`.

How it works: The name `ClassPathXmlApplicationContext` gives away its behavior. It tells the Spring framework to look for an XML configuration file residing in the application's **Classpath**.

Where is the Classpath? In a standard Maven project structure, any file placed inside the `src/main/resources` folder is automatically bundled into the root of the classpath when the application is compiled. Therefore, we right-click the `resources` folder and create a new file named `beans.xml`.

In our `Main` Java file, we boot up the container like this:

```
ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
```

Structuring the `beans.xml` File

An empty XML file is useless to Spring. Before we can start defining our Beans, we must define the XML Metadata headers (the schema). This schema dictates the rules of the XML file, ensuring we only use valid Spring XML tags.

Note: Developers almost never memorize this schema header. The standard practice is to copy it from the official Spring documentation and paste it at the top of the XML file.

Creating Your First XML Bean

With the schema in place, we define our very first Bean—the `OrderService`.

Instead of writing a Java method annotated with `@Bean`, we use the `<bean>` XML tag inside our `beans.xml` file.

To define a bean, Spring requires two pieces of critical information:

1. **id**: A unique identifier for the bean. The standard naming convention is to use the exact name of the Java class, but in camelCase (e.g., `orderService`).
2. **class**: The fully qualified path to the Java class, which includes the entire package structure (e.g., `in.stricks.OrderService`).

The XML Implementation:

```
<bean id="orderService" class="in.stricks.OrderService" />
```

(Notice we use a self-closing tag `</>` because there are no inner configurations required yet).

What happens at runtime? Because Spring Beans are eagerly initialized (created immediately upon startup) and act as Singletons by default, the moment `new ClassPathXmlApplicationContext("beans.xml")` executes, Spring reads this tag, locates the `in.stricks.OrderService` class, calls its default constructor, and holds that instance in the container under the ID `orderService`.

If we add a `System.out.println("Order Service Created");` to the default constructor of `OrderService`, we will immediately see it print to the console when the app runs, proving the object was successfully created purely via XML!

Feature	Annotation-Based (<code>@Bean</code> in <code>@Configuration</code>)	XML-Based (<code><bean></code> in <code>beans.xml</code>)
Location	Inside a Java class annotated with <code>@Configuration</code> .	Inside an XML file in the <code>resources</code> folder.
Identifier	Defaults to the method name.	Explicitly defined using the <code>id</code> attribute.
Type Definition	Defined by the return type of the Java method.	Explicitly defined using the fully qualified <code>class</code> package path.
Code Intrusion	High. Spring annotations live alongside business logic.	Zero. Java classes remain pure POJOs without Spring imports.

Retrieving Beans from the Container

Once the `ClassPathXmlApplicationContext` is initialized and the beans are created eagerly, we need a way to fetch them into our `Main` method to use them. In the XML configuration approach, the methods for retrieving beans remain identical to the annotation approach, but their behaviors have nuanced differences based on how the XML is structured.

There are three primary ways to fetch a bean from the context:

1. Get Bean By Type:

```
OrderService order = context.getBean(OrderService.class);
```

This method asks the container to find a bean that matches the `OrderService` class. It works perfectly as long as there is exactly *one* bean of this type defined in the `beans.xml` file. 2. **Get Bean By ID:**

```
OrderService order = (OrderService) context.getBean("orderService");
```

This method fetches the bean using the unique `id` defined in the XML. However, because the container only knows it's returning a generic `Object`, you must explicitly type-cast it to `OrderService`. 3. **Get Bean By ID and Type (The Best Approach):**

```
OrderService order = context.getBean("orderService", OrderService.class);
```

This approach is the most robust. By providing both the ID and the class type, you avoid manual type-casting, and you eliminate any ambiguity for the IoC container.

Retrieval Method	Pros	Cons
By Type	Clean syntax, no type-casting required.	Fails immediately if multiple beans of the same class exist in the XML.
By ID	Pinpoints the exact bean instance.	Requires manual type-casting; prone to <code>ClassCastException</code> if mismatched.
By ID and Type	Type-safe, no manual casting, handles multiple beans perfectly.	Slightly more verbose syntax.

Handling Multiple Beans (`NoUniqueBeanDefinitionException`)

A critical scenario arises if you define multiple beans of the exact same class in your `beans.xml` file. For example:

```
<bean id="orderService" class="in.stricks.OrderService" />
<bean id="orderService2" class="in.stricks.OrderService" />
```

This does *not* violate the Singleton design principle. Spring will simply create and manage two distinct Singleton instances of the `OrderService` class, each tracked by its unique ID.

However, if you attempt to fetch the bean **By Type** (`context.getBean(OrderService.class)`), the program will crash with a `NoUniqueBeanDefinitionException`. The IoC container becomes confused because it finds two matching beans and doesn't know which one you want. This is exactly why using the **By ID and Type** retrieval method is the industry standard—it explicitly tells Spring which of the two instances to fetch.

Bean Identifiers: `id` vs `name`

When defining a bean in XML, the `id` attribute is optional. If you omit it, the bean is still created, but it has no explicit identifier, meaning you can *only* fetch it by Type.

If you do use an `id`, it must be strictly unique. If two beans share the same `id`, the XML parser will throw a fatal error on startup.

However, Spring provides another attribute: `name`.

```
<bean id="orderService" name="orderServiceBean, osBean1, osBean2"
class="in.stricks.OrderService" />
```

The `name` attribute allows you to assign multiple **aliases** to a single bean, separated by commas or spaces. You can then use any of these aliases in the `getBean()` method.

- `id`: Strictly unique, used as the primary identifier.
- `name`: Can contain multiple aliases to reference the same underlying bean instance.

XML Dependency Injection: Constructor Injection

The true power of the Spring IoC container is not just creating objects, but wiring them together (Dependency Injection).

Imagine a scenario where `OrderService` is dependent on `PaymentService`. In the Java code, `OrderService` has a constructor that expects a `PaymentService` object:

```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

In the annotation world, we would rely on `@Autowired`. In the XML world, we must explicitly map this relationship using the `<constructor-arg>` tag.

How it works in XML: First, you define the independent bean (`PaymentService`). Then, inside the dependent bean's tag (`OrderService`), you use `<constructor-arg>` alongside the `ref` (reference) attribute to point to the ID of the independent bean.

```

<!-- 1. Create the independent bean -->
<bean id="paymentService" class="in.stricks.PaymentService" />

<!-- 2. Create the dependent bean and inject the reference -->
<bean id="orderService" class="in.stricks.OrderService">
    <constructor-arg ref="paymentService" />
</bean>

```



Syntax error in text mermaid version 12.0.0

Constructor Injection with Literal Values

Constructors don't just take other objects; they often take literal values like Strings or Integers. Suppose `PaymentService` has a constructor expecting a String `type` and an Integer `retryCount`.

Instead of the `ref` attribute, we use the `value` attribute:

```

<bean id="paymentService" class="in.stricks.PaymentService">
    <constructor-arg value="UPI" />
    <constructor-arg value="3" />
</bean>

```

Managing Argument Order: If a constructor has multiple arguments, mapping them based purely on vertical order can be brittle. To fix this, Spring XML provides two attributes to ensure values map to the exact right parameters:

1. **index:** Maps the value to the positional index of the constructor parameter (0-based).
2. **name:** Maps the value to the exact parameter name defined in the Java constructor.

```

<bean id="paymentService" class="in.stricks.PaymentService">
    <constructor-arg name="type" value="UPI" />
    <constructor-arg name="retryCount" value="3" />
</bean>

```

Using the `name` or `index` attribute allows you to write the XML tags in any order, guaranteeing that an integer doesn't accidentally get passed into a string parameter.

XML Dependency Injection: Setter Injection

The second form of dependency injection is Setter Injection. (Note: Field injection—where Spring directly injects into a private field—is **impossible** in XML configuration. XML configuration operates strictly from the outside and cannot bypass Java access modifiers to write to private fields. It requires either a public constructor or a public setter method).

If `OrderService` uses a setter method instead of a constructor to receive `PaymentService`:

```
public void setPaymentService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}
```

In the XML, instead of `<constructor-arg>`, we use the `<property>` tag.

```
<bean id="orderService" class="in.stricks.OrderService">  
    <property name="paymentService" ref="paymentService" />  
</bean>
```

The Magic of the `name` Attribute in `<property>`: The `name` attribute in the `<property>` tag is highly specific. It does **not** refer to the name of the private field in your Java class. It explicitly refers to the **Setter Method's name**, following standard JavaBeans naming conventions.

If the XML says `name="paymentService"`, Spring automatically capitalizes the first letter and prepends "set", searching the class for a method named `setPaymentService()`.

To prove this, if you change your Java method name to `setPaymentServiceBean()`, your XML will break unless you update the property tag to:

```
<property name="paymentServiceBean" ref="paymentService" />
```

Injection Type	XML Tag Used	Key Attributes	Requirements in Java Class
Constructor	<code><constructor-arg></code>	<code>ref</code> , <code>value</code> , <code>index</code> , <code>name</code>	A matching parameterized constructor.
Setter	<code><property></code>	<code>name</code> , <code>ref</code> , <code>value</code>	A public setter method following <code>setX()</code> naming convention.
Field	N/A	N/A	Cannot be done via XML.

Resolving Ambiguity: Interfaces and Polymorphism

In a real-world enterprise application, we rarely depend on concrete classes. Instead, we code to interfaces.

The Setup: Imagine modifying the architecture so that `PaymentService` is no longer a concrete class, but an interface. We then create two concrete classes that implement it: `UpiPaymentService` and `CardPaymentService`.

Our `OrderService` now expects a `PaymentService` object in its constructor.

The Problem: In the annotation-based approach, if both `UpiPaymentService` and `CardPaymentService` are annotated with `@Component`, the IoC container encounters a fatal conflict when trying to auto-wire `OrderService`. It finds two matching beans for the `PaymentService` type and throws a `NoUniqueBeanDefinitionException`. To fix this in Java, we had to rely on `@Primary` (to make one the default) or `@Qualifier` (to specify exactly which one to inject).

The XML Solution (Explicit Referencing): In XML configuration, this confusion is entirely bypassed if you use explicit references. You first define the two implementations as separate beans:

```
<bean id="upiPaymentService" class="in.stricks.payment.UpiPaymentService" />
<bean id="cardPaymentService" class="in.stricks.payment.CardPaymentService" />
```

Then, when configuring the `OrderService` bean, you explicitly provide the exact ID of the implementation you want injected into the constructor's `ref` attribute:

```
<bean id="orderService" class="in.stricks.OrderService">
  <constructor-arg ref="cardPaymentService" />
</bean>
```

Because you strictly provided the ID `cardPaymentService`, the IoC container completely ignores `upiPaymentService`. There is zero ambiguity.

(Note: XML does support a `primary="true"` attribute on the `<bean>` tag, which mirrors the `@Primary` annotation, but it is rarely necessary because explicit ID referencing solves the issue directly).

Configuring Bean Scopes (Singleton vs. Prototype)

By default, every bean defined in the XML file is created as a **Singleton** with **Eager Initialization** (instantiated the moment the application context loads).

If you require a new, distinct instance of an object every time it is requested, you must change its scope to **Prototype**.

How it works in XML: Instead of placing a `@Scope("prototype")` annotation above your Java class, you simply add the `scope` attribute to your `<bean>` tag.

```
<bean id="userService" class="in.stricks.UserService" scope="prototype" />
```

The Behavioral Shift: When the `ClassPathXmlApplicationContext` loads, it will parse this tag but will *not* instantiate the `UserService` object. The constructor is completely ignored at startup. The object is only created (lazily) when your Java code explicitly requests it via `context.getBean(UserService.class)`.

Scope	Initialization Timing	Instances Created	Use Case
Singleton (Default)	Eager (at Context Startup)	Exactly 1	Stateless services, Repositories, shared logic.
Prototype	Lazy (on <code>getBean()</code> request)	New instance per request	Stateful objects, temporary data holders.

Injecting Collections (Lists, Sets, Maps)

Constructors and setters do not just take single objects or primitive strings; they frequently require Collections. For instance, what if `UserService` expects a `List<String> usernames` in its constructor?

Passing a single `value="Aditya"` will fail because a String is not a List. Spring XML provides specialized inner tags to handle Java Collection Framework injections.

Injecting a List: Inside the `<constructor-arg>`, you nest a `<list>` tag, and inside that, you define individual `<value>` tags.

```
<bean id="userService" class="in.stricks.UserService">
  <constructor-arg name="usernames">
    <list>
      <value>Aditya</value>
      <value>Rohit</value>
      <value>Rohan</value>
    </list>
  </constructor-arg>
</bean>
```

Injecting a Map: If the constructor expected a `Map<String, Integer>`, you would use the `<map>` tag combined with `<entry>` tags defining the key-value pairs.

```
<map>
  <entry key="Admin" value="1" />
  <entry key="Guest" value="2" />
</map>
```

This capability proves that XML configuration is robust enough to handle complex data structures natively without requiring Java-side initialization logic.

Bean Lifecycle Callbacks (Init and Destroy Methods)

In a Spring Bean's lifecycle, developers often need to run custom logic immediately after the bean is created and all dependencies are injected (Post-Construct), or right before the bean is destroyed (Pre-Destroy).

In modern Spring, we use `@PostConstruct` and `@PreDestroy`. In XML, we map standard Java methods to these lifecycle events.

The Setup: We create two standard, unannotated public methods in our `UserService` class:

```
public void init() { System.out.println("Post-Construct Phase"); }  
public void cleanup() { System.out.println("Pre-Destroy Phase"); }
```

The XML Implementation: We inform the IoC container about these methods using the `init-method` and `destroy-method` attributes on the `<bean>` tag.

```
<bean id="userService" class="in.stricks.UserService"  
      init-method="init"  
      destroy-method="cleanup" />
```

Triggering the Destroy Method: The `init` method runs automatically upon creation. However, the `cleanup` method will not execute if the application abruptly stops. You must gracefully shut down the IoC container by calling `context.close()`. *(Crucial Note: As with annotation-based configuration, Spring does not manage the complete lifecycle of Prototype scoped beans. If your bean is `scope="prototype"`, the `destroy-method` will never be called by the container).*

Managing Large Configurations: Splitting and Importing XML Files

In legacy enterprise applications, a single `beans.xml` file can quickly grow to thousands of lines, becoming unmaintainable.

To solve this, Spring allows you to break your configuration into multiple logical XML files (e.g., `payment-beans.xml`, `user-beans.xml`). You then create one master configuration file (e.g., `appConfig.xml`) that acts as an aggregator using the `<import>` tag.



Syntax error in text mermaid version 12.0.0

In your Java `Main` class, you only load the master file:

```
ApplicationContext context = new ClassPathXmlApplicationContext("appConfig.xml");
```

The IoC container reads the imports, merges all the bean definitions from the disparate files, and builds a single, unified context map.

The Hybrid Approach: Merging XML and Annotations

In reality, massive codebases are rarely rewritten overnight. When migrating a legacy application from XML to Annotations, developers use a hybrid approach.

The IoC container is entirely agnostic to the metadata source. You can use a single `ClassPathXmlApplicationContext` and include a specific tag in your XML (`<context:component-scan base-package="in.stricks" />`).

When Spring reads this file, it will instantiate the explicitly defined XML beans, but it will also scan the specified Java packages for `@Component` annotations and instantiate those beans as well. All of them will live harmoniously inside the same IoC container and can be injected into one another seamlessly.

Actionable Takeaways

- **Audit Legacy Code Confidently:** If you encounter a `ClassPathXmlApplicationContext`, know that the application relies on an XML file (usually in the `resources` folder) for its bean wiring, not Java annotations.
- **Use ID + Class for Bean Retrieval:** Always use `context.getBean("beanId", TargetClass.class)` to fetch beans from the container. It prevents `NoUniqueBeanDefinitionException` when multiple beans share a class, and avoids dangerous manual type-casting.
- **Leverage Explicit Referencing for Polymorphism:** When an interface has multiple implementations, bypass ambiguity by explicitly injecting the exact bean ID into the dependent bean's `<constructor-arg ref="specificBeanId" />`.
- **Verify Setter vs. Constructor Injection Rules:** If you are wiring via XML, remember that `<constructor-arg>` requires a matching Java constructor, while `<property>` strictly maps to a `setX()` method. Field injection is impossible in pure XML.
- **Graceful Context Shutdown for Cleanup:** If you define a `destroy-method` in your XML (or use `@PreDestroy`), ensure your application runner explicitly calls `context.close()` to guarantee that resources (like database connections or open files) are properly released.

